

CS0.101: Computer Programming

Quiz 1 (Fall 2022)

Duration: 45 mins

Question 1: (5 marks)

Write a program to find the sum of first n odd numbers. Here n can be taken to be an input from the user.

Question 2: (5 marks)

Arrange the following in decreasing order of space taken: `int`, `char`, `double`, `short`. Also mention the space taken by each of these data types.

Question 3: (5 marks)

Describe the One's complement and Two's complement method of storing negative numbers. What advantage does the Two's complement method have over One's complement method?

Question 4: (5 marks)

Predict the output of the following code fragment assuming it is inside a valid `main()` function

```
int i=256, j=256;
i >>= 8;
j <<= 24;
printf("%d %d\n", i, j);
```

j left

Question 5: (5 marks)

Spot errors(if any) in the following code fragments:

1. This fragment is supposed to print $x = 0$.

```
int x=5;
while(x >0);
    x/=2;
printf(x=%c\n", x);
```

2. `float a=12.25, b=12.52;`

```
if(a = b)
    printf("a and b are equal");
```

$x = x/2$

4

(1)

Question 6:

What is output of the following programs with proper justification ?

1. (5 marks)

```
#include<stdio.h>
int main()
{
    int x=4, y, z;
    y = - - x;
    z=x - -;
    printf("%d %d %d\n", x, y, z);
    return 0;
}
```

Handwritten notes: $y = 3; x = 3$
 $z = 3; x = 2$

2. (5 marks)

```
#include<stdio.h>
int main()
{
    int p, q, r;
    p = 8 > 5 > 2;
    q = 8 > 5 > 0;
    r = 8 > 5 > 1;
    printf("%d %d %d\n", p, q, r);
    return 0;
}
```

3. (5 marks)

```
#include<stdio.h>
int main()
{
    int i=1, j=1;
    for (;;)
    {
        if (i > 5)
            break;
        else
            j += i;
        printf("%d\n", j);
        i += j;
    }
    return 0;
}
```

Handwritten notes: $j = 2$
 $i = 2$

4. (5 marks)

```
#include<stdio.h>
int main()
{
    int p, q, r;
    p = 9;
    q = 10;
    r = p == q;
    printf("%d\n", r);
    return 0;
}
```

Question 7: (5 marks)

Write a program to print "Hello world" without using the semicolon ";" anywhere in the program.

CS0.101: Computer Programming (Section A)

Quiz 1 (Fall 2023)

Duration: 45 mins

Question 1: (5 marks)

Write a program to find the factorial of a number. Here n can be taken to be an input from the user.

Question 2: (5 marks)

What is type-casting? Explain the types of type casting in C.

Question 3: (5 marks)

Write -35 in 2's complement form (assume 8-bit representation).

Question 5: (4 marks)

State True/False with justification: Given two positive integers x and y , it is not possible to calculate the minimum and maximum of the two numbers without using any conditions or loops.

Question 6: (10 marks)

Write a program to print the following pattern of the Pascal's triangle.

```
      1
     1 1
    1 2 1
   1 3 3 1
```


CS0.101: Computer Programming (Section B)

Quiz 1 (Fall 2023)

Duration: 45 mins

Question 1: (5 marks)

Write a program to find the sum of first n even numbers. Here n can be taken to be an input from the user.

Question 2: (5 marks)

Explain the control flow between the RAM, Hard disk and the CPU after a program is written.

Question 3: (5 marks)

Write -15 in 1's complement form (assume 8-bit representation).

Question 4: (5 marks)

State True/False with justification: Given two numbers x and y , the expression $y \wedge ((x \wedge y) \oplus (x < y))$ calculates the minimum of the two numbers.

Question 5: (10 marks)

Write a program to print the following pattern.

```
      1
     2 3
    4 5 6
   7 8 9 10
```

`int a=1`

`for (int n=1; n<5, n++) {`

`for (int i=1; i<=n, i++)`

`print(" ");`

`for (int i=1; i<=n, i++)`

`printf("%d ", a)`

`a++;`

Question 6:

What is output of the following programs with proper justification ?

✓ 1. (5 marks)

```
#include<stdio.h>
int main()
{
    int x=7, y, z;
    y = -- x;
    z = x + +;
    printf("%d %d %d\n", x, y, z);
    return 0;
}
```

✓ 2. (5 marks)

```
#include<stdio.h>
int main()
{
    int i;
    for(i = 9; i!=0; i - -)
        printf("i= %d\n", i - -);
    return 0;
}
```

✓ 3. (5 marks)

```
#include<stdio.h>
int main()
{
    int x = 4, y = 4, z = 4;
    if (x == y == z)
    {
        printf("YES");
    }
    else
    {
        printf("NO");
    }
    return 0;
}
```

4. (5 marks)

```
#include<stdio.h>
int main()
{
    printf("%d", printf("Hello World"));
    return 0;
}
```

1 Short answer questions

1. Assuming 8 bits, express 33 in 2's complement notation. (5 marks)

Solutions: First, write the positive number 33 in binary. 33 in binary (base 2) is 00100001. Since 33 is a positive number and its 2's complement will be the same as its binary representation.

2. What is the space taken by each of the following data types: char, double, int, float, short? (5 marks)

Solution: char: 1 byte, double: 8 bytes, int: 4 bytes, float: 4 bytes, short: 2 bytes.

3. What is the output of the following program? Answer with justification. (5 marks)

```
1      int i = 2, j = 2, k = 2;
2      i += j += k;
3      printf("%d %d %d", i, j, k);
```

Answer: output: 6 4 2

Justification:

- k remains 2 as it was not modified.
- j is updated to 4 because $j+ = k$ where k was 2.
- i is updated to 6 because $i+ = j$ after j was updated to 4.

4. What is the output of the following program? Answer with justification. (5 marks)

```
1      int n=0;
2      if (n >= 1 <= 10)
3          printf("n is between 1 and 10\n");
```

Answer: n is between 1 and 10

Justification: inside if condition, first it will check either $n \geq 1$ (which is false, 0 in C) then $0 \leq 10$. The condition does not check if n is between 1 and 10; it is a miswritten condition that still results in true due to how the operators are processed.

2 Multiple choice questions

In all questions, you can assume that the code is inside the body of a main function. Put a tick mark on the correct answer only.

Question 1: (3 marks)

What is the output of the following program?

```
1    int num = 2;
2    printf("%d", (num << 1) + (num >> 1));
```

- (A) 2
- (B) 1
- (C) 4
- (D) 5

Answer: D, output 5

Solution: Lets break down the expression:

- $num \ll 1$: This shifts the binary representation of num (which is 10 in binary) one bit to the left. This results in 100 in binary, which is equivalent to 4 in decimal.
- $num \gg 1$: This shifts the binary representation of num one bit to the right. This results in 01 in binary, which is equivalent to 1 in decimal.
- $(num \ll 1) + (num \gg 1)$: This adds the results of the two shift operations, 4 and 1, together. This gives us 5.

Therefore, the output of the code will be 5.

Question 2: (3 marks)

What is the output of the following program?

```
1    int i = 0, j = 0;
2    for (i = 0; i < 100; i++) {
3        for (j = 0; j < 1;) {
4            break;
5        }
6        printf("CProgramming\n");
7    }
```

- (A) CProgramming is printed 1 time.
- (B) Nothing is printed.
- (C) CProgramming is printed 100 times.

(D) Error

Answer: C, output: CProgramming ... (printed 100 times).

Solution:

- The inner loop doesn't affect the output directly. It's entered and immediately exited for each iteration of the outer loop.
- The break in the inner loop only exits that inner loop, not the outer one.
- The printf statement is executed once for each iteration of the outer loop.
- The variable j is reset to 0 at the start of each outer loop iteration, but it's not used meaningfully in this code.

This code structure is unusual and not typically how nested loops would be used efficiently. The inner loop and the j variable don't serve a practical purpose in this example.

Question 3: (3 marks)

What is the output of the following program?

```
1    int a=2, b=3;
2    printf("%d %d", a/b, b/a);
```

- (A) 0 1
- (B) 0.666666 1.5
- (C) 0.66 1.500000
- (D) None of these

Answer: A, output: 0 1

Solution:

- Integer division in C always results in an integer. The result is truncated, not rounded. This means any fractional part is simply discarded.
- If you need precise division with fractions, you should use floating-point numbers (float or double) instead of integers.
- The order of operands matters in integer division when the numbers aren't evenly divisible.

Question 4: (3 marks)

What is the output of the following program?

```

1    int i, j, count;
2    count=0;
3    for(i=0; i<5; i++);
4    {
5        count++;
6    }
7    printf("%d",count);

```

- (A) 1
 (B) 5
 (C) 0
 (D) None of these

Answer: A, output: 1

Solution:

- The semicolon after the for loop is the crucial detail here. It creates an empty loop body.
- The block with count++ is not repeated; it executes just once after the loop.
- This is a common mistake in C programming, where an unintended semicolon can significantly change the program's behaviour.

Question 5: (3 marks)

What is the output of the following program?

```

1    int i = 0, j = 0;
2    while (i<5 & j<10) {
3        i++;
4        j++;
5    }
6    printf("%d %d", i, j);

```

- (A) 5 5
 (B) 5 10
 (C) Error
 (D) 10 10

Answer: A, output: 5 5

Solution:

- While loop stops when i reaches 5, regardless of j's value.
- Both i and j are incremented the same number of times, so they end up with the same value.

- The use of & instead of && doesn't affect the outcome in this case, but it's generally not recommended. && is the proper logical AND operator in C.

Question 6: (3 marks)

What is the output of the following program?

```

1    int i = 3;
2    printf("%d\n", i++);
3    printf("%d\n", i++);

```

- (A) 4 4
 (B) 3 4
 (C) 3 3
 (D) 4 5

Answer: B, output: 3 4)

Solution: The postfix increment operator (i++) uses the current value of i in the expression, and then increments it. If we had used prefix increment (++i) instead, the output would have been different (4 and 5). After these operations, the final value of i is 5, although this isn't printed.

This code demonstrates the difference between the value of a variable and the side effects of operators. The postfix increment ensures that the "old" value is used in the expression before the increment takes place.

Question 7: (3 marks)

What is the output of the following program?

```

1    int a = 2;
2    switch(a) {
3        case 1: printf("1 ");
4        case 2: printf("2 ");
5        case 3: printf("3 ");
6        default: printf("None");
7    }

```

- (A) 2.
 (B) None.
 (C) 1 2 3 None.
 (D) None of these.

Answer: D, output: 2 3 None

Solution:

- a is initialized to 2.
- The switch statement checks the value of a.

- Since a is 2, execution starts at case 2:.
- There are no break statements after each case, so once a matching case is found, all subsequent cases are executed until the end of the switch block or until a break is encountered.
- With break statements, only "2" would be printed for $a = 2$.

Question 8: (3 marks)
What is the output of the following program?

```

1      int x = printf("Hello!");
2      printf("%d", x);

```

- (A) 5
(B) Error
(C) 1
(D) None of these

Answer: D, output: Hello!6

Solution: This code demonstrates an interesting feature of printf: it returns the number of characters it prints. This can be useful for error checking or, in this case, to determine the length of the output string.

Question 9: (3 marks)

What is the value of x, y, z after the following program (assuming the initial values are $x = 0, y = 0, z = 1$)?

```

1      if (x)
2          if (y)
3              if (z)
4                  z = 3;
5              else
6                  z = 2;

```

- (A) 0 0 3
(B) 0 0 2
(C) 0 0 1
(D) Error

Answer: C, output: 0 0 1

Solution:

- First, it checks if x is true (non-zero in C, here $x = 0$ means false) and will not execute the nested if.

- If x is true, it checks if y is true (here $y = 0$ means false).
- If both x and y are true, it checks if z is true.
- If x, y , and z are all true, it sets $z = 3$.
- If x and y are true, but z is false, it sets $z = 2$.

Question 10: (3 marks)
What is the output of the following program?

```

1      double x = 0;
2      for (x = 0.0; x < 5.0; x++);
3          printf("%lf\n", x);

```

- (A) 0.000000 1.000000 2.000000 3.000000 4.000000
(B) 4.000000
(C) 5.000000
(D) Error

Answer: C, output: 5.000000

Solution: The program prints 5.000000 because the loop increments x from 0.0 to 5.0, and when x reaches 5.0, the loop condition ($x < 5.0$) fails, leaving x with the value 5.0. Since ';' after the for loop, 'printf' will execute only once. x value is then printed.

CPro-M25 Quiz 1 Rubrik

Q1 - 5 Marks

Explanation - 56 in signed 8 bit Binary = `00111000`, taking two's complement, first flip all the bits to get `11000111` and add 1 to get `11001000`

[2] for writing `56 = 00111000`

[5] for all correct bits - `11001000`

[0] for anything else

Q2 - 3 Marks

```
printf("%d", ((x ^ y) < 0));
```

Explanation - Since sign is decided by the MSB, if both the integers are of same sign (same MSB), MSB will be 0 after xor, hence $> 0 \Rightarrow$ outputs 0. If the two integers are of different sign (different MSB), MSB will be 1 after xor, hence $< 0 \Rightarrow$ outputs 1

[1] for `computes xor of the two values and compares it with 0, if  $\geq 0$ , prints 0, if  $< 0$ , prints 1`

[1] If assumed the operator as subtraction and written `outputs 0 if  $x \geq y$ , outputs 1 if  $x < y$`

[3] for `Outputs 0 if both the values are of same sign, outputs 1 if different sign`

[0] for anything else

Q3 - 3 Marks

1. for (i = 0; i < 10 ; i++)
2. for (; i < 10 ; i++)
3. for (i = 0; ; i++)
4. for (i = 0; i < 10 ;)
5. for (; ;)

Explanation - All for loops are valid, even though some fields are empty, the syntax is fine since there are semicolons.

[1] for ticking **option 1**

[+0.5] per option ticked from **options 2-5**

Q4 - 3 Marks

```
int i = 1, j;  
for ( ; ; ) {  
    if (i)        j = --i;  
    if (j < 10)    printf("CproQuiz", j++);  
    else          break;  
}  
return 0;
```

Explanation - First iteration j is set to 0, and then on, everytime **CproQuiz** is printed and j is incremented until it hits 10 when it breaks.

[1] for **CproQuiz**

[3] for **CproQuizCproQuizCproQuizCproQuizCproQuizCproQuizCproQuizCproQuizCproQuizCproQuiz** i.e CproQuiz repeated **10** times. No space or new line between each CproQuiz string.

[-1] if newline is omitted

[0] for anything else

Q5 - 3 Marks

```
int i, count;
count=0;

for(i=0; i<2025; i++); {
    count++;
}

printf("%d", count);
```

Explanation - Notice the semicolon after the for loop before the { braces. The for loop won't execute anything. After for loop, count will be incremented once.

[1] for 2025

[3] for 1

[0] for anything else

Q6 - 3 Marks

```
int p = 1;
while (_____) {
    int last = arr[0];
    for (int i = 0; ____; i++) {
        arr[i] = arr[i + 1];
    }
    _____
    p++;
}
```

Explanation - p starts from 1 and since we have to do cyclic shift d times, $p \leq d$ is the condition in while, which is the first fill in the blank. For the condition inside for

loop, variable `i` shouldn't be `n-1`, since then `arr[i+1]` will result in RTE. Hence `i < n-1`, i.e only go till `i = n-2`, which is the second fill in the blank. To do a cyclic shift, we need to update the last element with the 0th element, hence `arr[n-1] = last` which would be the answer for the third.

[1] `p ≤ d`

[+1] `i < n-1`

[+1] `arr[n-1] = last;`

[0] for anything else

Binary marking for each fill in the blank. All `<`, `<=` signs should be correct. `;` should be included in the last answer.

Q7 - 3 Marks

```
char x = 6;  
char y = ~ x << 2;  
  
printf("%d\n", y);
```

Explanation - 6 in binary is `00000110` and since `~` has higher precedence than `<<` (given), `~x` is computed first which is `11111001`. Shifting all bits 2 positions to the left and adding 0s, we get `11100100`. Taking 2's complement to convert to decimal, we get `00011100` which is 28. Adding negative sign, since MSB is 1, we get `-28`

[1] for `~x = ~6 = ~00000110 = 11111001`

[+1] for `~x << 2 = 11100100`

[+1] for converting `11100100` to decimal which is `-28`

[3] for writing `-28`

[0] for anything else

Q8 - 3 Marks

```
signed char a = 100, b = 30;  
signed char c = a + b;  
  
printf("a+b = %d\n", c);
```

Explanation - When storing 130 i.e 100+30 in a signed char, it is stored with 1 in the MSB. C always stores signed datatypes in 2's complement form. Converting `130 = 10000010` from 2's complement form to decimal, the value becomes `-126` when we convert it to decimal.

[1] for `100+30 = 10000010` in 8-bit binary

[+1] for taking two's complement `01111110`

[+1] for converting to decimal and adding - sign to get `-126`

[3] for `-126` . No extra characters need to be checked.

[0] for anything else

Q9 - 3 Marks

```
int x = 0, y = 5;  
  
if (x++ && ++y)  
    printf("Branch 1\n");  
  
printf("x = %d, y = %d\n", x, y);  
  
if (++x || y++)  
    printf("Branch 2\n");
```

```
printf("x = %d, y = %d\n", x, y);
```

Explanation - Since it is post increment, `x++` returns 0, and hence Branch1 is not printed. `++y` is not executed because of short circuiting, but `x++` updates `x` to 1. After this, `x` is 1 and `y` remains same i.e 5. After this, in the second if condition, `++x` returns 2, hence Branch 2 is outputted, but short circuit happens again, hence `y++` is not executed again. `x` is incremented to 2 and `y` remains 5 which is the final output.

[+1] for `x = 1, y = 5`

[+1] for writing Branch 2

[+1] for `x = 2, y = 5`

[-1] if Branch 1 is written

[0] if total marks for this ques < 0 return 0;

Q10 - 3 Marks

```
int a = 3, b = 7, c = 2;
if (a++ > 3) {
    if (--b < 7) {
        printf("P5: X\n");
    } else {
        printf("P5: Y\n");
    }
} else {
    if (c++ == 2) {
        printf("P5: Z\n");
    } else {
        printf("P5: W\n");
    }
}
```

Explanation - First condition evaluates to False since it is post increment operation, and $a = 3$ is not > 3 . In the second else condition, $c++ == 2$ evaluates to True since it is post increment again i.e $c++$ returns 2 and $2 == 2$, Hence, it prints P5: Z

[3] for P5: Z

[0] for any other option

Q11 - 3 Marks

```
int i, j;
for (i = 1; i <= 3; i++) {
    for (j = i; j <= 3; j++) {
        printf("%d ", i + j);
    }
    printf("\n");
}
```

Explanation - i goes from 1 to 3, while j goes from i to 3. For $i=1$, j goes from 1 to 3 and $i+j$ is printed as 2 3 4 for each j . For $i=2$, j goes from 2 to 3 and 4 5 is printed on a new line and for $i=3$, $j = 3$, hence $i+j = 6$ is printed on a new line.

2 3 4

4 5

6

[+1] for each line of correct output. Note the spacing and new lines.

[0] for wrong formatting or wrong output

[-1] if new line is omitted

Q12 - 3 Marks

```

int a[] = {4, 3, 5, 2, 1};
int i = a[0];
while (i != 1) {
    printf("%d ", i);
    i = a[i - 1];
}

```

Explanation - i starts with `a[0] = 4` which is printed first. After this, `i = a[i-1] = a[4-1] = a[3] = 2` and hence 2 is printed next. After this, `i = a[i-1] = a[2-1] = a[1] = 3` and hence 3 is printed next. After this, `i = a[i-1] = a[3-1] = a[2] = 5` and hence 5 is printed next. Now, `i = a[i-1] = a[5-1] = a[4] = 1` and it doesn't print 1, it breaks out of the for loop.

[1] if `4` is written as the first output

[3] for all correct outputs - `4 2 3 5`

[-1] if `1` is printed as the last integer.

[-1] if newline is omitted

[0] for anything else or marks in this question < 0